JAVA 2

Module TAB

Tableaux, boucles et opérateur ternaire

Au programme

1. Un tableau, une taille fixe
2. Parcourir un tableau
3. Des lignes inégales
4. L'opérateur ternaire
5. Des objets qui se connaissent

1. Un tableau, c'est une taille fixe



Démonstration



Démonstration

Cinq pièges, et pourquoi ils existent

Déroulé : Demo_TAB

Déclarer un tableau

La taille est décidée au new, et ne change plus

Les collections, qui grandissent, viendront au module COL

```
int[]    nombres = new int[5];              // 5 cases, toutes a 0
String[] mots    = { "Hello", "world" };    // 2 cases, deja remplies
```

Ce que la démonstration a montré

Un tableau de `int` naît à zéro, un tableau d'objets à `null`

`System.out.println(tableau)` affiche une adresse, pas des valeurs

Il faut `Arrays.toString(tableau)`

`tableau.length` est un **champ**, `texte.length()` une **méthode**

Passer un tableau à une méthode, c'est passer son adresse

Un tableau de tableaux

Un tableau à deux dimensions n'existe pas vraiment en Java

C'est un tableau dont chaque case contient un autre tableau

```
int[][] matrice = new int[3][3];   // un tableau DE TABLEAUX

matrice[1]                         // la deuxieme ligne, un int[]
matrice[1][2]                      // la troisieme case de cette ligne
```

2. Parcourir un tableau



La boucle indexée

On dispose de l'indice, donc on peut écrire dans le tableau

```
for (int i = 0; i < tab.length; i++) {
    tab[i] = tab[i] * 2;
}
```

La boucle `for-each`

Lecture seule : la variable est une copie de la case

Plus courte, plus lisible, et sans risque de déborder

```
for (int v : tab) {
    System.out.println(v);
}
```

Laquelle choisir

Vous lisez seulement ⟶ `for-each`

Vous écrivez dans le tableau ⟶ boucle indexée

Vous avez besoin de la position ⟶ boucle indexée

Dans le doute, `for-each` : c'est celle qui se relit le mieux

Deux boucles imbriquées

La boucle extérieure parcourt les lignes, l'intérieure les cases

Un compteur qui traverse les deux se déclare **avant** la première

```
int valeur = 1;
for (int ligne = 0; ligne < n; ligne++) {
    for (int col = 0; col < n; col++) {
        matrice[ligne][col] = valeur;
        valeur = valeur + 1;
    }
}
```

Exercices

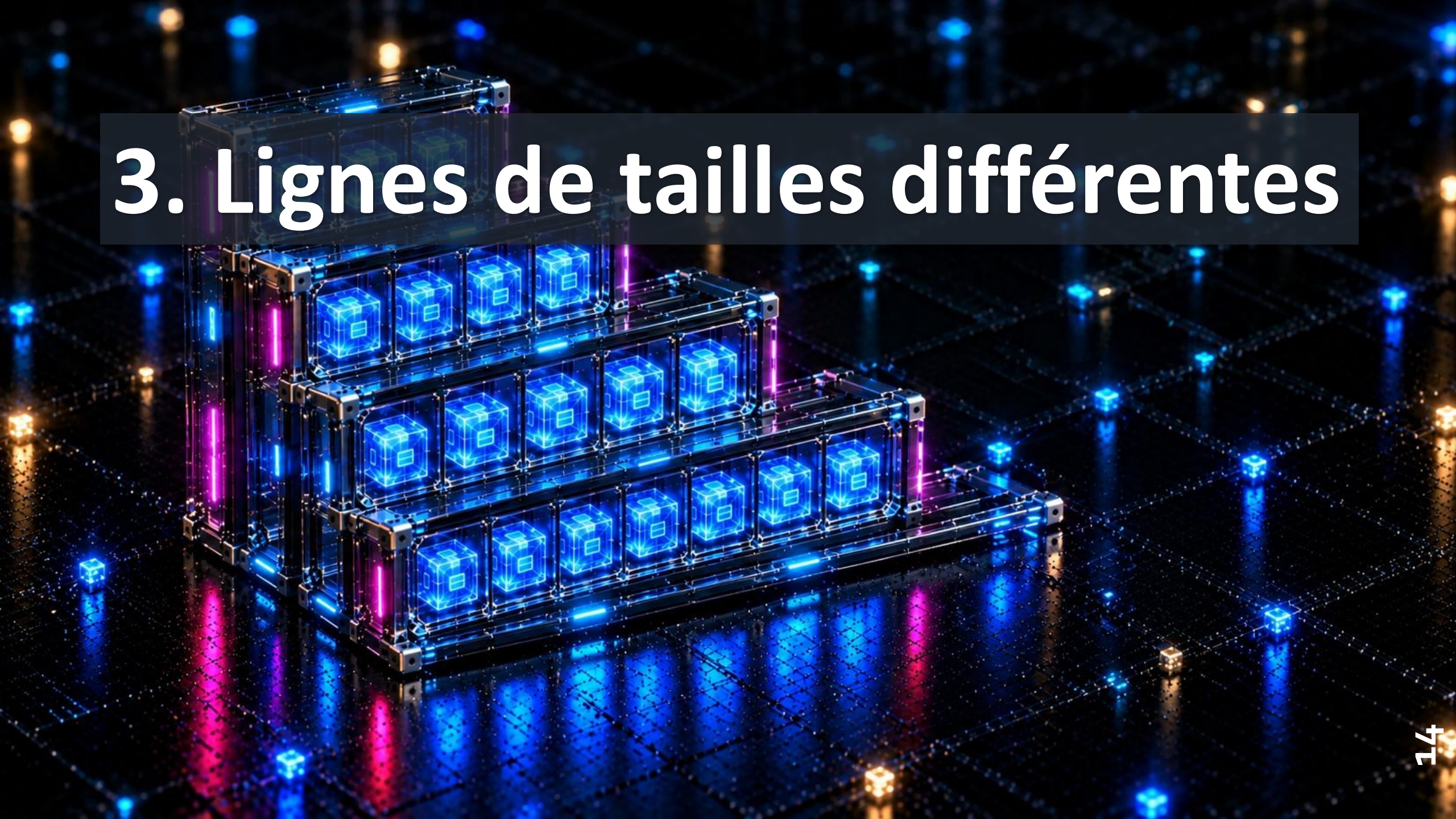


TAB.1 Une matrice carrée

fascicule TAB

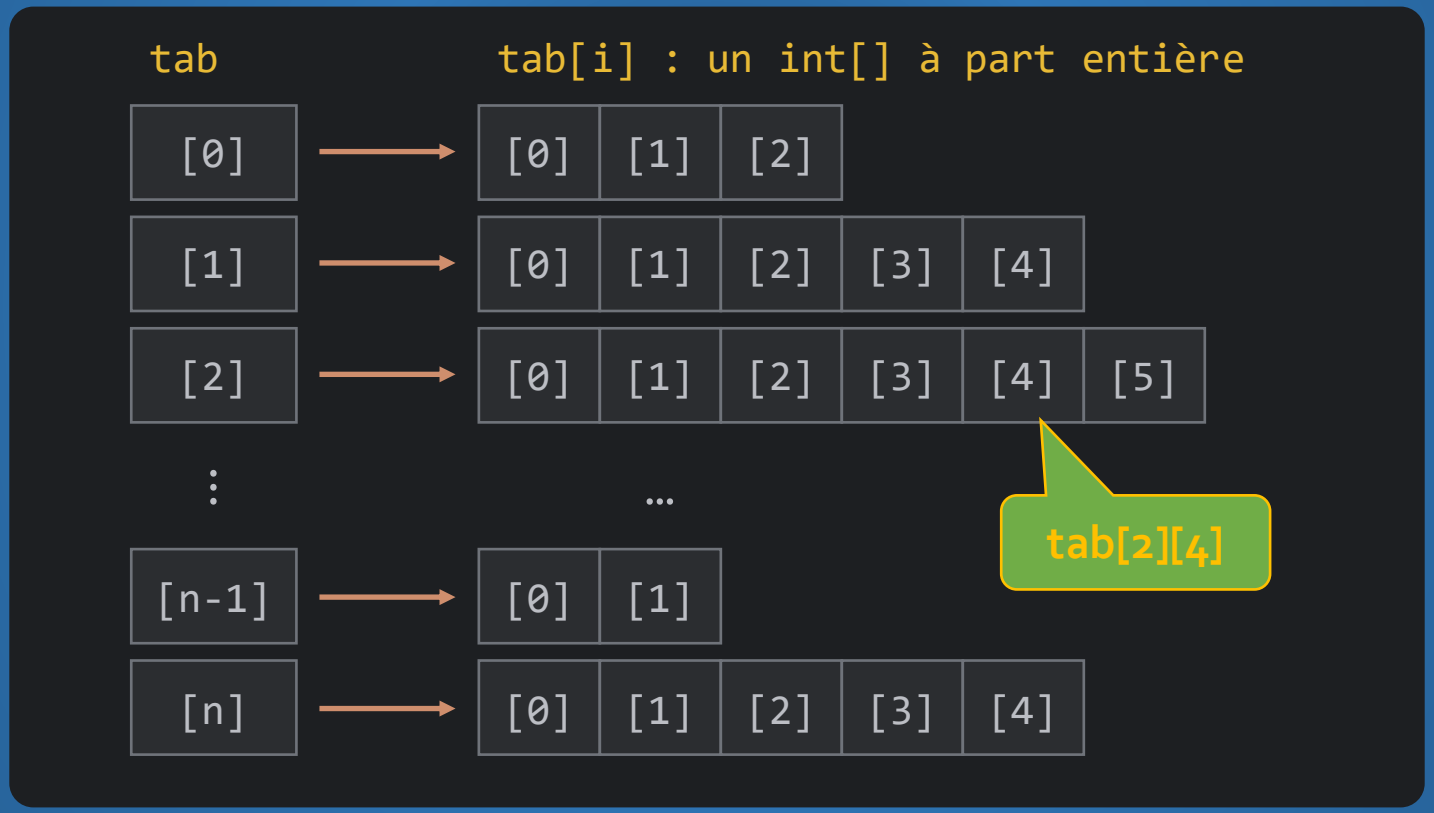
`carree` construit, `enTexte` met en forme, aucune des deux n'affiche

3. Lignes de tailles différentes



Un tableau de tableaux, en image

Chaque case contient un tableau, de la longueur qu'on veut



Déclarer les lignes une par une

`new int[n][]` : le nombre de lignes seulement

Chaque ligne reçoit ensuite sa propre taille

Tant qu'une ligne n'est pas créée, elle vaut `null`

```
int[][] triangle = new int[n][];

for (int i = 0; i < n; i++) {
    triangle[i] = new int[i + 1];
}
```

Exercices



TAB.2 Le triangle de Pascal

fascicule TAB

Deux boucles `for-each` imbriquées pour l'affichage

4. L'opérateur ternaire



La syntaxe

Une condition, deux valeurs

```
condition ? valeurSiVrai : valeurSiFaux

public int max(int a, int b) {
    return a > b ? a : b;
}
```

Ce qu'un `if` ne permet pas

Le ternaire **retourne une valeur** ; le `if` ne retourne rien

Il se place donc à droite d'un `return` ou d'une affectation

```
int plusGrand = a > b ? a : b;      // possible
int plusGrand = if (a > b) a; ...   // n'existe pas
```

Un booléen se retourne tel quel

`a > b` est déjà un booléen : il se retourne tel quel

Le `if`, et même le ternaire `? true : false`, n'y ajoutent rien

```
// trop verbeux
if (a > b) {
    return true;
} else {
    return false;
}
```

```
// direct
return a > b;
```

Où s'arrête l'intérêt

Le ternaire s'imbrique : `a ? x : (b ? y : z)`

Écrivez-en un à trois niveaux, relisez-le, puis décidez

Des méthodes courtes et lisibles - **CC-MET.2**

Exercices



TAB.3 L'opérateur ternaire

fascicule TAB

Deux méthodes, une instruction chacune

5. Des objets qui se connaissent



Un objet peut en contenir un autre

Un champ peut être un objet, comme il peut être un `int`

Ici, le vendeur est un rôle que joue une personne : il **a une** `Person`

On aurait pu dire qu'il **est une** personne, par héritage, comme en C# : c'est un autre modèle, que le module `ABS` compare à celui-ci

```
Person  <---[1]---  Vendor  ---[*]--->  Sale  ---[1]--->  Car
```

Ne pas stocker ce qui se déduit

Le prix réel se calcule, il ne se range pas

Un champ en plus, c'est une donnée qui peut se désynchroniser

Même problème que le champ `age` de `POO.3`

Un tableau de taille fixe dans un objet

Un tableau ne grandit pas : il faut décider d'un maximum

Et retenir soi-même combien de cases sont occupées

`enregistrer` retourne `false` quand le tableau est plein

Parcourir jusqu'au nombre enregistré, pas jusqu'à `ventes.length`

Exercices



TAB.4 Des objets qui se connaissent

fascicule TAB

Quatre classes, un format de texte imposé

Ce qu'il faut retenir

Un tableau a une **taille fixe**, décidée au `new`

Un tableau est une **référence** : le passer, c'est passer son adresse

`matrice[1]` est une ligne, `matrice[1][2]` est un nombre

`for-each` pour lire, boucle indexée pour écrire

Le ternaire retourne une valeur ; un `if` non

Ce qui se déduit ne se stocke pas